

Big Data Analytics with Spark MLlib

Big Data Analysis

Introduction to Spark Machine Learning

Machine learning is becoming increasingly commonplace in many industries as companies use it to personalize their customers' website experiences, leverage sophisticated visual and speech recognition capabilities, derive insight from industry trends, predict future business opportunities, etc. Because the volume of data that many companies have had to handle has increased dramatically in recent years, data scientists have often found themselves spending more time dealing with the underlying technology infrastructure than with the actual analyses. Fortunately, Spark can help reverse this trend with a general machine learning library called MLlib, which is not only scalable but also provides easy integration with a variety of other tools. By using MLlib, analysts can spend more of their time conducting analyses.

Before we walk through a few examples of how machine learning models can be used to make predictions using Spark, we will look at several fundamental machine learning concepts.

What is Machine Learning?

Simply put, machine learning is a form of data analysis. Using algorithms that continuously learn from data, machine learning allows computers to recognize hidden patterns without actually being programmed to do so. The key aspect of machine learning is that as models are exposed to new data sets, they adapt to produce reliable and consistent output.

What is Driving the Resurgence of Machine Learning?

There are three interrelated phenomena that are behind the growing prominence of Machine Learning: 1) the ever-increasing volume, variety and velocity of data, 2) the decrease in bandwidth and storage costs, and 3) the exponential improvements in computational processing. In a nutshell, the ability to perform complex mathematical computations on big data is driving the resurgence in machine learning.

Commonly Used Methods in Machine Learning

Supervised Machine Learning

In *supervised machine learning*, algorithms are trained using *labeled* examples, which means that the desired output for a given input is known. For instance, a piece of mail could be labeled either as *relevant* or *junk*. The algorithm is then provided with a set of inputs (perhaps a sampling of the content) along with the corresponding correct output (whether it's considered relevant or junk) to foster learning. Once the algorithm is trained on a set of labeled data, it is run against the same labeled data and its actual output is compared with the correct output to detect errors. If errors are discovered, the model is refined to minimize those errors. Learning stops when the algorithm achieves an acceptable level of performance. The algorithm is then applied to an *unlabeled* data set to make predictions about data it hasn't seen before. Specifically, it attempts to predict whether new pieces of mail are relevant or junk. (See Figure 1 for a schematic representation of this process.)

Supervised learning is commonly applied to situations where historical data can be used to predict future outcomes. Examples of practical applications include:

- Document classification
- Image detection
- Topic classification
- Churn analysis
- Sales forecasting
- Housing price predictions
- Stock market predictions

Now, let's take a brief look at two of the more frequently used techniques in supervised machine learning, *regression analysis* and *classification analysis* (as implemented with logistic regression).

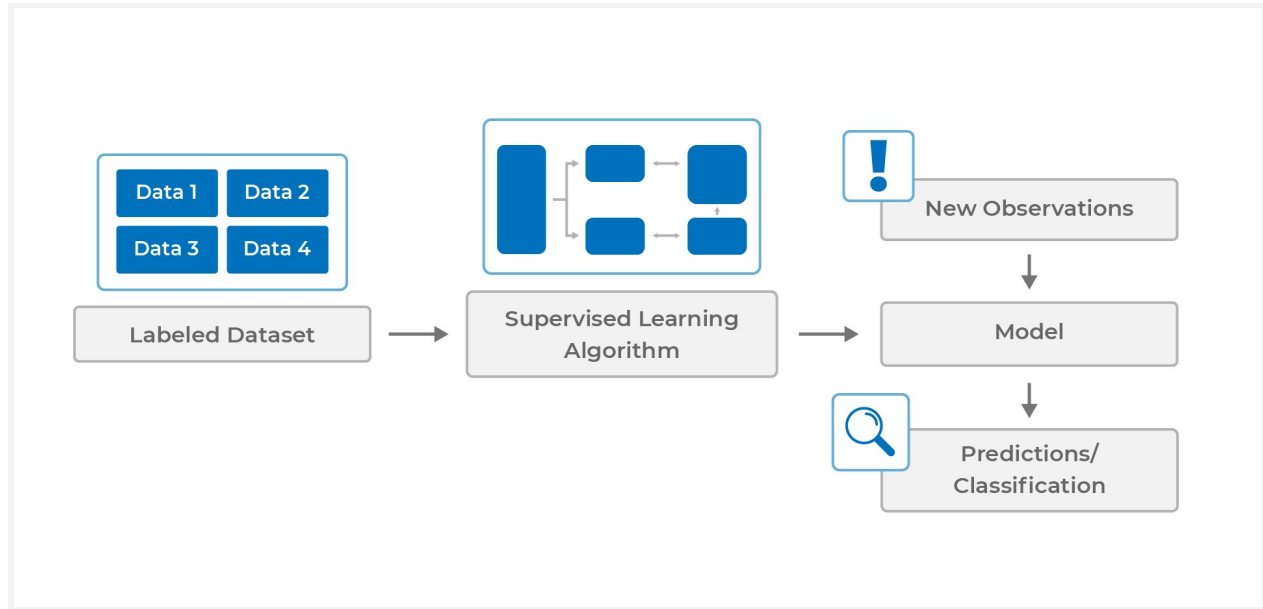


Figure 1: A schematic representation of the supervised machine learning process

Overview of Linear Regression

In the context of machine learning, *linear regression* is a classical supervised machine learning model used to predict values of continuous variables given one or more input values. For example, the model can predict future housing prices based on prevailing market prices, predict anticipated stock market fluctuations based on recent trends, and so forth. Essentially, linear regression mathematically models the linear relationship between a *dependent* or *target* variable and one or more *independent* or *explanatory* variables. Keep in mind when working with this kind of model that there is no implication of causality. In other words, one variable does not necessarily “cause” the other one.

The scatter plot in Figure 2 shows a set of data having a dependent variable (plotted along the y-axis) and an independent variable (plotted along the x-axis).

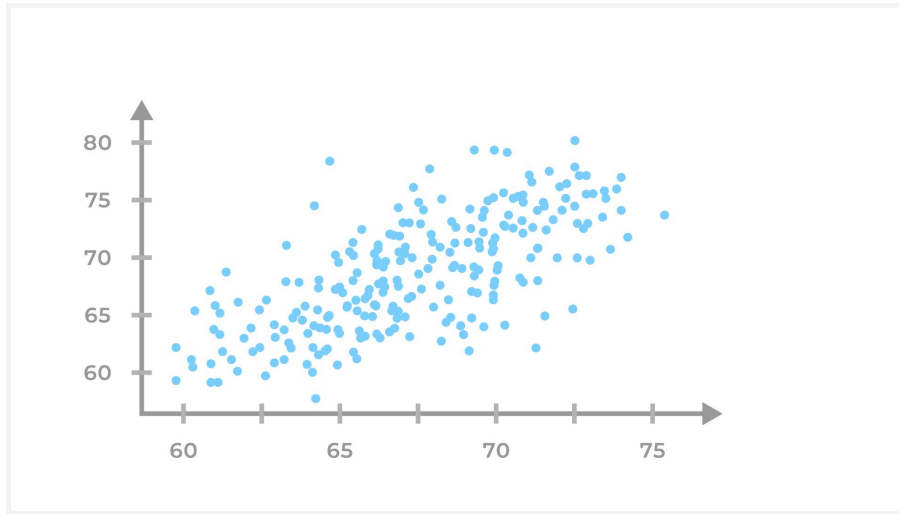


Figure 2: A scatter plot showing a set of sample data

The objective of linear regression is to “fit” a straight line through the data points such that the vertical distances between the points and the line (called *residuals*) are all minimized (Figure 3). There are several ways of achieving this, the most frequently used of which is the *method of least squares*.

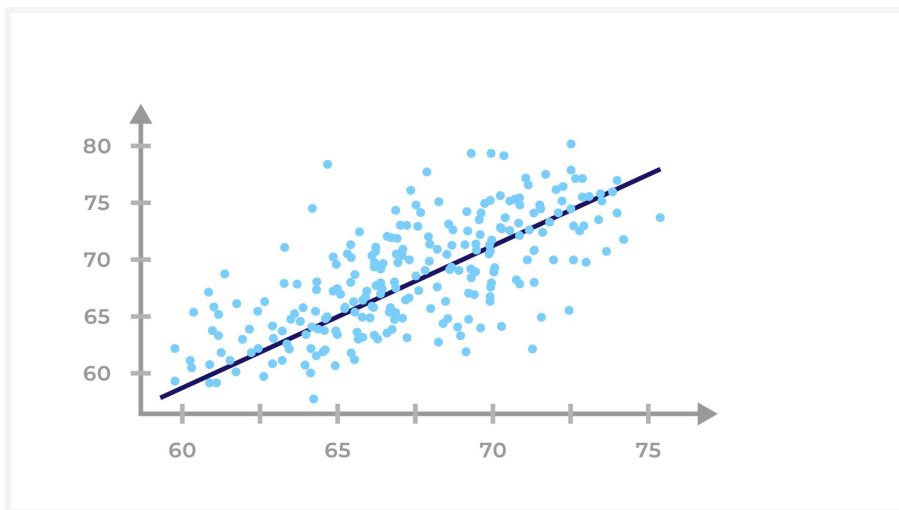


Figure 3: A scatter plot showing sample data along with a best-fitting regression line

The least squares method entails minimizing the sum of squares of the residuals (Figure 4).

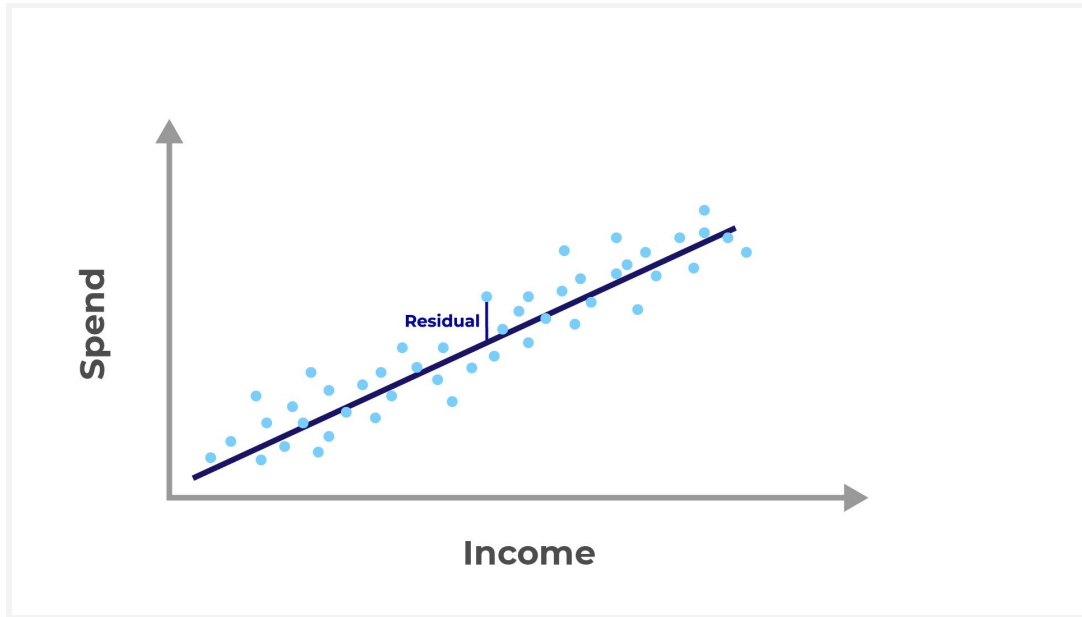


Figure 4: A best-fitting linear regression line with the residual for one data point indicated

As an example, suppose you have a set of observations consisting of the yearly amount each family in town spends on vacation plotted as a function of the families' income. The residuals consist of the difference between a family's actual expenditure and the corresponding value on the regression line, so the least squares method determines the equation of the line that minimizes the squares of those residuals.

For the mathematically inclined reader, the equation of the "best fit" regression line is of the form

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n$$

Evaluating Regression Performance

The *coefficient of determination*, represented as r^2 (r-squared), is a measure of how well the fitted line represents the data. A larger value of r^2 represents a better fit, which implies that the model can better explain the variability in the output based on the inputs.

If the regression equation fits the data perfectly (i.e., the data points all lie along a line), the sum of squares would be zero and r^2 would be equal to one. The values of actual and predicted values would be identical.

Overview of Logistic Regression

Logistic regression is a special flavor of linear regression in which the dependent variable is categorical. We'll start by looking at *binary* dependent variables in which the values are coded as a one or a zero, representing two possibilities such as yes or no, on or off, etc.

Mathematically, logistic regression starts with the logistic function

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n)}}$$

where p is the probability that the output variable is 1.0 and is depicted graphically by the s-shaped curve in Figure 5. Because we're dealing with probability, the curve has a maximum value of one (when x goes to infinity) and a minimum value of zero (when x goes to negative infinity).

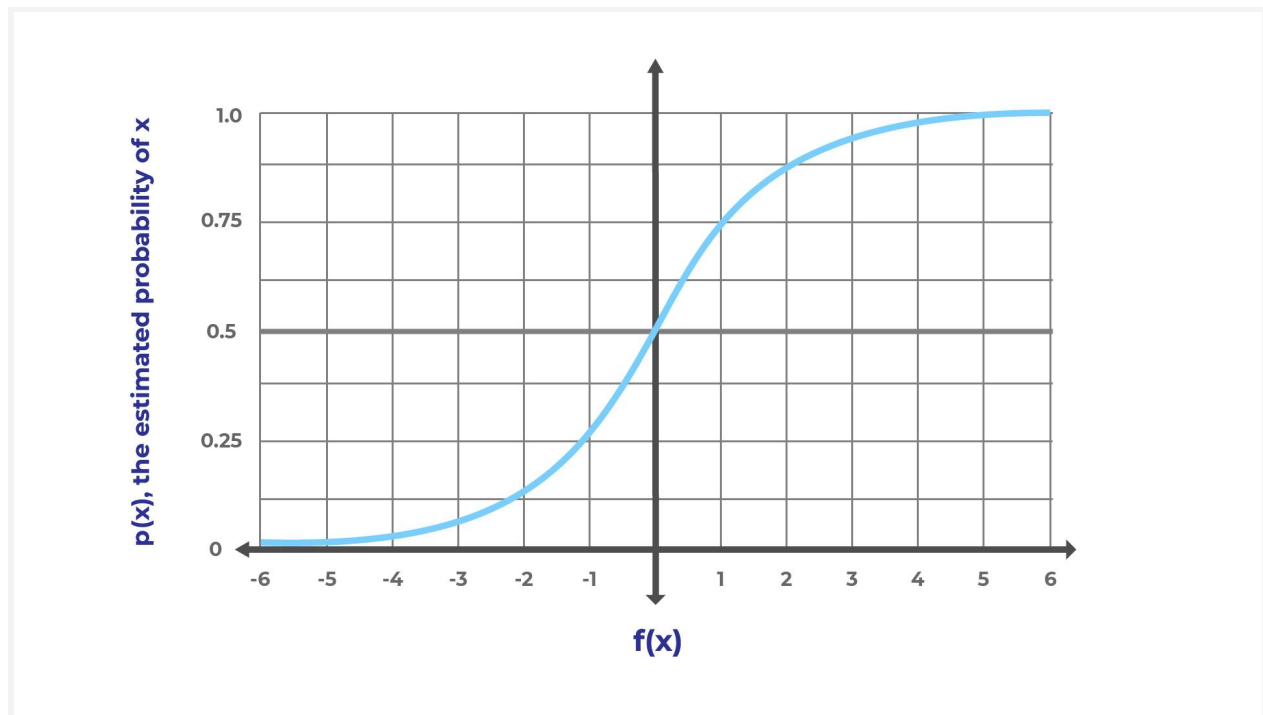


Figure 5: The s-shaped curve depicting the estimated probability of x as given by the logistic function

We interpret the output $p(x)$ of this function as follows:

- If $p(x) > 0.5$, the outcome is considered to be a one (or *yes*, or *true*, etc.)
- If $p(x) < 0.5$, the outcome is considered to be a zero (or *no*, or *false*, etc.)

For example, suppose we are trying to predict customer churn and the logistic function gives us $p(x) = 0.75$ for a particular customer. This means that there is a 75 percent chance that the customer will churn.

Types of Logistic Regression

So far, we have looked at *binary logistic regression* in which there are only two possible outcomes (one/zero, true/false, yes/no, churn/not churn, etc.).

However, there are two additional types that are commonly used in machine learning:

Multinomial logistic regression: The target variable has three or more *nominal* categories used for such situations as predicting the brand of dishwasher soap a customer is likely to buy or the type of automobile a customer is likely to drive.

Ordinal logistic regression: The target variable has three or more *ordinal* categories such as a product rating from 1 to 5.

Unsupervised Machine Learning

In *unsupervised machine learning*, there are no “right answers” available to train the system (Figure 6). The algorithm must explore the data and detect hidden patterns or structures within the data. Unsupervised machine learning works well with transactional data because, for example, it can identify a group of individuals having similar purchasing behaviors that can then be treated as a single homogenous unit for the purposes of marketing promotions. Two commonly used techniques in unsupervised machine learning are *K-means clustering* and *association modeling*.

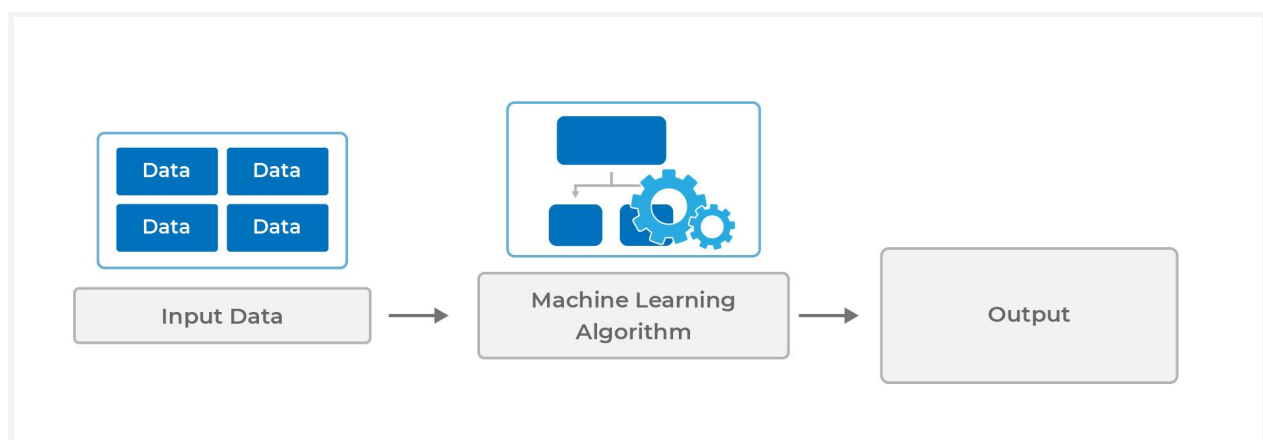


Figure 6: A schematic representation of the unsupervised machine learning process

Applications of unsupervised learning algorithms include:

- Market basket analysis

- Recommendation systems
- Spam detection
- Automatic topic extraction

Overview of K-Means Clustering

Clustering is an unsupervised learning model that groups similar items (in the underlying data) together into clusters.

Typical examples of clustering include:

- Clustering customers together based on purchasing behaviors or patterns
- Grouping similar documents together
- Grouping together any items that have similar characteristics

The overall objective is to divide a set of data into distinct groups such that all the members of a group have similar characteristics.

Figure 7 shows two plots consisting of the same data points with different clusters indicated by different colors. The plot on the left shows two possible clusters based on “croissant-like” shapes while the plot on the right shows ten possible clusters based on data point proximity.

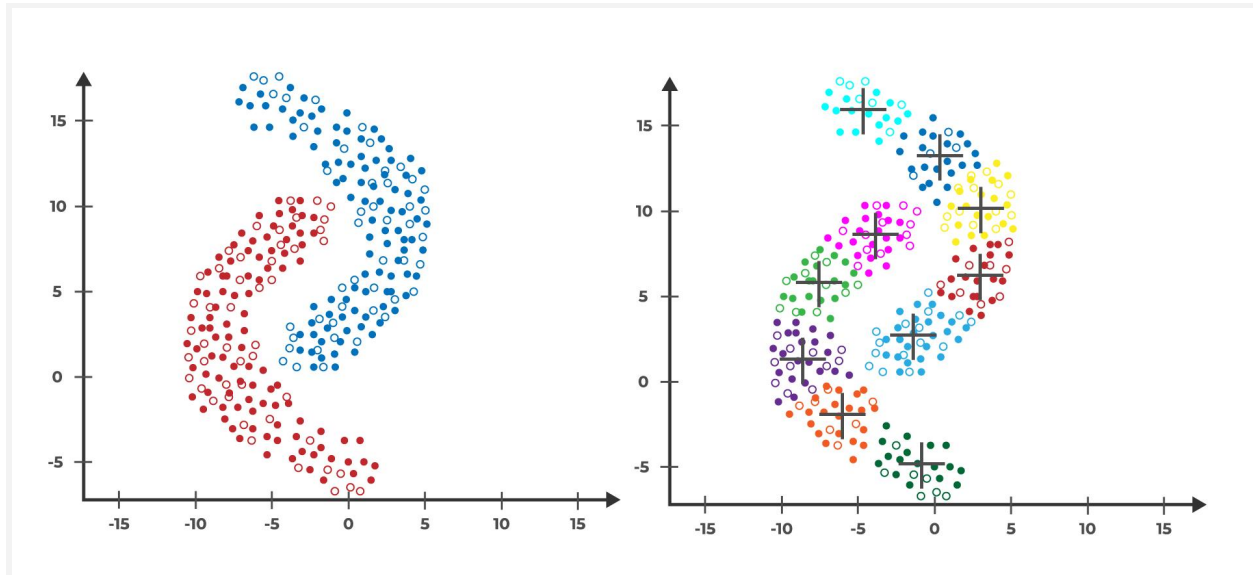


Figure 7: x-y plots showing data points grouped into clusters

One of the predominant clustering techniques is called *K-means clustering*. The K-means algorithm attempts to group data points into K clusters or groups using the following process:

1. A number K of “center” points (or *means*) are selected at random locations in the data domain. (See, for example, the plus signs in the right-hand plot of Figure 7.)
2. For every data point:
 - a. The distance between each data point and each of the center points is calculated.
 - b. Each data point is assigned to a cluster of points all having the same nearest center point.
3. After the clusters have been identified in step two above, new center points (means) for the clusters are calculated based on the actual distribution of points in each cluster.
4. Steps 2 and 3 are repeated until no data point changes its cluster membership.

Choosing a “K” Value

There is no easy way to determine the correct or optimal number of clusters. One approach is to use the “elbow method,” which entails choosing a value for K and determining the clusters as described above. Next, you find the distance between each data point and its cluster center, square those distances, and then sum them. The resulting value is called the *sum of square distances*. This is repeated for several different values of K , such as 2, 4, 6, 8, etc.

Now you plot the sum of squared distances as a function of k (the number of clusters) as shown in Figure 8. A good value for K (an optimal number of clusters) occurs where the sum of squared distances decreases suddenly (the “elbow” in the plot).

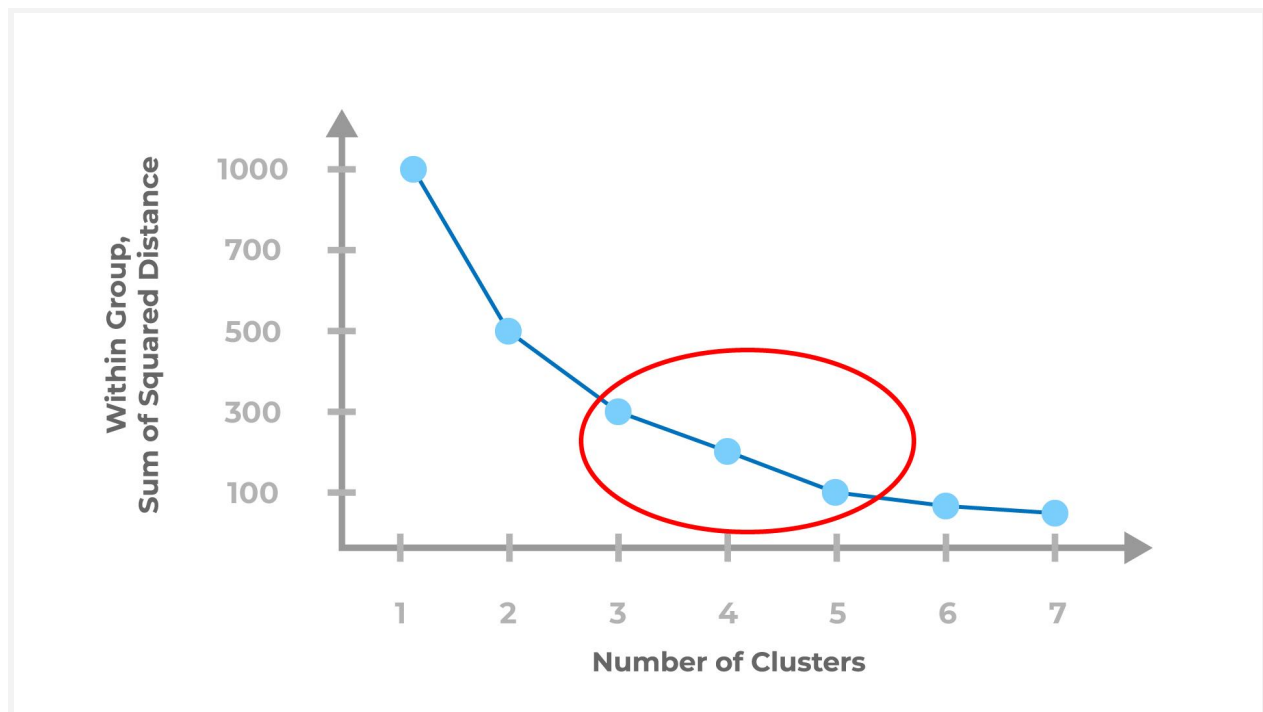


Figure 8: Sum of squared distance values as a function of the number of clusters in a dataset. The optimal number of clusters corresponds to the point where the sum of squared distances suddenly drops.